A MINI PROJECT
REPORT ON

FORECASTING NETFLIX STOCK PRICES WITH LSTM: MACHINE LEARNING INSIGHTS

Submitted in partial fulfillment of the requirement

for the award of the degree of

BACHELOR OF

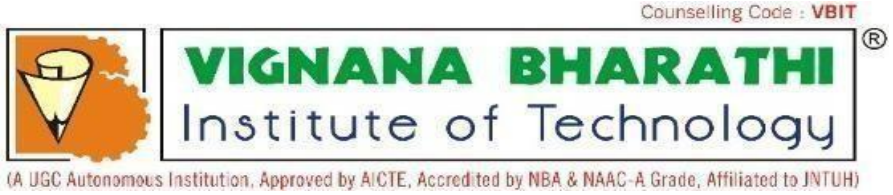
TECHNOLOGY IN

COMPUTER SCIENCE AND BUSINESS SYSTEM

By

B. LUCKY HEMANTH RAJ	**21P61A3207**
CH.SRI NAGA SANTOSH	**21P61A3212**
J. SAI AKANKSHA	**21P61A3224**
K. INDRANI	**21P61A3226**

Under the esteemed guidance of
Mr. MAHENDER S
ASSISTANT PROFESSOR
Dept. of CSBS



VIGNANA BHARATHI INSTITUTE OF TECHNOLOGY

(A UGC Autonomous Institution, Approved by AICTE, Affiliated to JNTUH,

Accredited by NBA & NAAC) Aushapur (V), Ghatkesar (M), Medchal(dist.)

September - 2024

Counselling Code : VBIT

VIGNANA BHARATHI Institute of Technology

(A UGC Autonomous Institution, Approved by AICTE, Accredited by NBA & NAAC-A Grade, Affiliated to JNTUH)

Aushapur (V), Ghatkesar (M), Hyderabad, Medchal – Dist, Telangana – 501 301.

DEPARTMENT
OF
COMPUTER SCIENCE & BUSINESS SYSTEM

CERTIFICATE

This is to certify that the mini project titled ***"FORECASTING NETFLIX STOCK PRICES WITH LSTM: MACHINE LEARNING INSIGHTS"*** *submitted by* ***B. LUCKY HEMANTH RAJ*** (21P61A3207), ***CH. SRI NAGA SANTOSH*** (21P61A3212), ***J. SAI AKANKSHA*** (21P61A3224), ***K. INDRANI*** (21P61A3226) *in B.tech IV-I semester Computer Science & Business System is a record of the bonafide work carried out by them.*

The results embodied in this report have not been submitted to any other University for the award of any degree.

INTERNAL GUIDE
Mr. Mahender S

HEAD OF THE DEPARTMENT
Dr. G. Swamy

EXTERNAL EXAMINER

DECLARATION

We, **Lucky Hemanth Raj, Ch. Sri Naga Santhosh, J. Sai Akanksha, and K. Indrani** bearing hall ticket numbers **(21p61a3207, 21p61a3212, 21p61a3224, 21p61a3226)** hereby declare that the mini project report entitled **"Forecasting Netflix Stock Prices with LSTM: Machine Learning Insights"** under the guidance of ***Mahender S,*** Assisstant Professor, Department of Computer Science and Business System**, Vignana Bharathi Institute of Technology, Hyderabad**, have submitted to Jawaharlal Nehru Technological University Hyderabad, Kukatpally, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Business System.

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

B. Lucky Hemanth Raj (21p61a3207)
Ch. Sri Naga Santhosh (21p61a3212)
J. Sai Akanksha (21p61a3224)
K. Indrani (21p61a3226)

ACKNOWLEDGEMENT

We are extremely thankful to our beloved Chairman, **Dr. N. Goutham Rao** and Secretary**, Dr. G. Manohar Reddy** who took keen interest to provide us the infrastructural facilities for carrying out the project work.

We wholeheartedly thank **Dr. P. V. S. Srinivas, Principal**, and **Dr. G. Swamy**, Head of the Department of Computer Science and Business Systems for their encouragement, support, and guidance in the project.

We would like to express our indebtedness to the project coordinator, **Mrs. K. Keerthana,** Assistant Professor in the Department of Computer Science and Business System, for her valuable guidance during the project work.

We thank our Project Guide, **Mahender S - Assistant Professor**, for providing us withan excellent project and guiding us in completing our mini project successfully.

We would like to express our sincere thanks to all the staff of Computer Science and Business System, VBIT, for their kind cooperation and timely help during the course of our project. Finally, we would like to thank our parents and friends who have always stood by us wheneverwe were in need of them.

ABSTRACT

This project aims to predict Netflix's stock price using historical data and a Long Short-Term Memory (LSTM) neural network model. We analyzed financial data from Yahoo Finance over the past 5000 days. This data includes daily open, high, low, and close prices, and volume, which are input features to train the LSTM model. To begin with, we visualized Netflix's historical price movement using a candlestick chart. The chart represents the stock's price patterns and trends over time. We also calculated the correlation between the closing price and other features (e.g., open price, volume), identifying significant relationships that may influence the stock's price movements. The LSTM model was chosen for its ability to capture long-term dependencies in sequential data. The data was preprocessed by splitting it into training and testing sets and reshaped to fit the LSTM model's input format. The LSTM model was constructed with two layers of LSTM units, followed by dense layers to predict the stock's closing price. After training the model with historical data, we evaluated its performance by predicting the closing price for a specific day, based on given input features. This approach demonstrates the potential of using deep learning models like LSTM for stock price prediction, providing insights for financial decision-making. Future improvements could involve hyperparameter tuning, incorporating additional technical indicators, and experimenting with different deep-learning architectures to enhance prediction accuracy.

Keywords: Stock price prediction, LSTM, Netflix, financial data, Yahoo Finance, candlestick chart, deep learning, neural networks, time series forecasting, technical analysis, machine learning, stock market, correlation analysis, sequential data, model training, financial forecasting.

TABLE OF CONTENTS

CHAPTER **PAGE NO.**

List of Figures

CHAPTER – 1

1. INTRODUCTION

1.1 INTRODUCTION TO THE SYSTEM

Stock price prediction is a challenging task due to the inherent volatility and complexity of the financial markets. Traditional statistical methods often fall short of capturing the intricate patterns within stock price data. However, machine learning models, particularly Long Short-Term Memory (LSTM) networks, have shown promise in improving prediction accuracy. This system focuses on forecasting Netflix's stock prices using an LSTM model. LSTM, a type of Recurrent Neural Network (RNN), is specifically designed to handle sequential data and can remember long-term dependencies, making it well-suited for time series forecasting. By leveraging historical stock data, the system aims to identify trends and patterns that influence Netflix's stock price movements. The LSTM model is trained on past data to learn these patterns, which are then used to predict future stock prices. This approach offers deeper insights into market trends and enhances the accuracy of stock price predictions, helping investors make more informed decisions. The system comprises key components such as data collection, preprocessing, model development, training, prediction, and evaluation, each contributing to the overall goal of accurate stock price forecasting using advanced machine-learning techniques. In recent years, LSTM models have gained significant traction in financial forecasting due to their ability to model complex, non-linear relationships over time. Unlike traditional models that struggle with the temporal nature of stock prices, LSTM networks are capable of learning both short-term fluctuations and long-term dependencies in the data. This makes them a powerful tool for capturing the intricate dynamics of stock price movements. By implementing an LSTM-based forecasting system for Netflix stock prices, this approach not only provides a more sophisticated analysis but also opens up possibilities for applying similar techniques to other stocks, enhancing predictive capabilities across various markets. The insights gained from such models can be instrumental for investors and financial analysts in devising more effective trading strategies.

1.2 PROBLEM STATEMENT

Stock price prediction is a highly challenging task due to the dynamic and volatile nature of financial markets. Accurate forecasting of stock prices is crucial for investors, analysts, and financial institutions, as it can significantly impact investment strategies and decision-making processes. Traditional statistical models, while useful, often struggle to capture the complex and non-linear relationships inherent in stock price data. These models tend to overlook the temporal dependencies

and intricate patterns that influence stock movements, leading to less reliable predictions.

Netflix, a global leader in the streaming industry, has seen its stock prices fluctuate due to various factors, including market trends, earnings reports, industry competition, and global events. Predicting Netflix's stock prices is particularly important given its influence on the broader entertainment and technology sectors. However, the volatility and unpredictability of its stock create a need for more sophisticated forecasting models.

The problem at hand is to develop a machine learning-based system that can accurately predict Netflix's stock prices by leveraging historical data. Specifically, the objective is to utilize Long Short-Term Memory (LSTM) networks, a specialized type of Recurrent Neural Network (RNN) that excels in time series forecasting. LSTM models are designed to handle sequences of data and can effectively learn both short-term fluctuations and long-term dependencies, making them suitable for predicting stock prices.

The challenge is to create an LSTM-based system that can outperform traditional forecasting methods by better capturing the temporal patterns in Netflix's stock price data. This system aims to provide more accurate and reliable stock price predictions, offering valuable insights for investors and enhancing decision-making in the financial market.

1.3 OBJECTIVE

The objective of this project is to develop a machine learning-based system that provides accurate forecasts of Netflix's stock prices using Long Short-Term Memory (LSTM) networks. LSTM models are particularly well-suited for this task due to their ability to handle sequential data and capture both short-term fluctuations and long-term dependencies, which are essential for reliable stock price prediction. Traditional forecasting methods often struggle to model the complex, non-linear relationships inherent in stock price data, leading to less accurate predictions. By employing LSTM networks, this project aims to address these limitations and enhance prediction accuracy.

The project involves several critical steps: first, collecting and preprocessing historical stock data from Netflix to prepare it for analysis. This includes normalizing the data, handling missing values, and transforming it into a format suitable for LSTM input. Next, an LSTM model will be built and optimized to learn from the preprocessed data. This involves configuring the model's architecture, tuning hyperparameters, and training the model to recognize patterns and trends that drive stock price movements.

To assess its accuracy, the model's performance will be evaluated by comparing its predictions with

actual stock prices. The ultimate goal is to provide a more reliable tool for forecasting Netflix's stock prices, offering valuable insights for investors and financial analysts to make informed decisions. Furthermore, this approach has the potential to be applied to other stocks and financial markets, thereby improving financial forecasting capabilities on a broader scale.

1.4 AIM OF THE PROJECT

This project aims to develop an accurate forecasting system for Netflix's stock prices using Long Short-Term Memory (LSTM) networks. By leveraging LSTM's capabilities to capture complex patterns in time series data, the project seeks to enhance prediction accuracy beyond traditional methods. It involves preprocessing historical stock data, building and optimizing the LSTM model, and evaluating its performance. The goal is to provide actionable insights for investors and analysts while establishing a framework applicable to other stocks and financial markets.

CHAPTER - 2

2. LITERATURE SURVEY

Survey of Stock Price Prediction of Netflix Using Machine Learning

- **Authors**: A. Gopala Krishna, K. Tanuja, G. Pallavi, G. Tarun Kalyan
- **Journal**: *International Journal of Innovative Science and Research Technology*, May 2023
- **Methodology**: This study utilizes machine learning techniques like Support Vector Machines (SVM) and Recurrent Neural Networks (RNN) for predicting Netflix's stock price. These models aim to capture the trends in stock movements based on historical data.
- **Key Findings**: The study highlights the efficacy of machine learning models in stock prediction but emphasizes the risks of overfitting and limited external factors affecting model predictions. It also notes challenges in model interpretability.
- **Gaps**: The study lacks sufficient incorporation of external market influences, raising concerns over generalizability. Additionally, the interpretability of the model outputs remains challenging.

Netflix Stock Price Movements: Insights from Data Mining

- **Authors**: V. Gowri, B. Harish, Fahad Ahmed, M. Srinath
- **Journal**: *IEEE 2nd Mysore Sub Section International Conference*, 2022
- **Methodology**: The paper employs data mining techniques, including decision trees and various predictive models, to gain insights into Netflix's stock price movements. Machine learning algorithms are used to enhance stock price forecasting.
- **Key Findings**: The study successfully demonstrates the application of machine learning and decision trees in stock price prediction but underscores the need for a wider data scope and improved model reliability.
- **Gaps**: The limited data scope raises concerns about prediction accuracy, and model overfitting remains an unresolved issue.

Stock Prediction Using ARIMA

- **Authors**: Rishabh Sharma, Rajeev Sharma, Shanmugasudaram Hariharan
- **Journal**: *International Conference on Computational Intelligence and Computing Applications (ICCICA)*
- **Methodology**: This research focuses on using the ARIMA (AutoRegressive Integrated Moving Average) model for stock prediction. It highlights the computational efficiency and predictive accuracy of ARIMA for time series data.
- **Key Findings**: The paper demonstrates that ARIMA can be effective in forecasting stock prices but suggests that hybrid models might enhance predictive performance. The study also mentions the potential limitations of the ARMA model in volatile market conditions.

- **Gaps**: The paper lacks an evaluation of comparative models in more complex market scenarios, and ARIMA's limitations under such conditions are noted.

Analysis of a Stock Exchange and Future Prediction Using LSTM

- **Authors**: B. Shivani, S. Govinda Rao
- **Journal**: *4th International Conference on Recent Trends in Computer Science and Technology*
- **Methodology**: This study explores Long Short-Term Memory (LSTM) networks as part of deep learning to predict stock prices. The use of both machine learning and deep learning models is highlighted in forecasting stock trends.
- **Key Findings**: The adoption of LSTM for stock price prediction shows promising results in capturing temporal dependencies. However, the study reveals the challenge of lacking interpretability and scalability of the models, especially in broader markets.
- **Gaps**: The model does not account for external.

2.1 EXISTING SYSTEM

Predicting stock prices is inherently complex, requiring a multifaceted approach that combines technical analysis, fundamental analysis, and market sentiment analysis. Machine learning algorithms play a pivotal role in analyzing historical stock market data to identify trends and patterns, which can then be used for prediction. These algorithms, including regression models, neural networks, and support vector machines, leverage past data to forecast future stock movements. However, it's essential to recognize that stock prices are influenced by a diverse array of factors such as market trends, economic indicators, company performance, and global events. Consequently, while machine learning models offer valuable insights, they may not fully capture all variables affecting stock prices.

In practice, existing systems for stock price prediction employ a range of machine learning techniques. These systems typically start by collecting extensive historical data on stock prices, supplemented by other relevant information such as market trends, economic indicators, and news articles related to the company. This comprehensive data is then input into machine learning models, which are trained to recognize patterns and make predictions based on historical trends. For instance, systems using neural networks, like LSTM models, can handle sequential data and identify complex patterns in stock prices over time.

Despite the advanced capabilities of these systems, it is important to acknowledge that stock price prediction remains an imperfect science. The accuracy of predictions can vary significantly due to the myriad of influencing factors and inherent market uncertainties. Therefore, while machine

learning can enhance forecasting, it should not be the sole basis for investment decisions. Investors are advised to exercise caution, consider additional analyses, and consult financial advisors before making substantial investment decisions based on these predictions.

2.2 PROPOSED SYSTEM

The proposed system involves collecting and preprocessing Netflix stock data, followed by applying machine learning algorithms like KNN regression to predict stock prices based on historical trends. The system also includes data cleaning, feature selection, and transformation for accurate predictions

Data Collection: The dataset is collected from Kaggle, covering four years (from February 5, 2018, to February 4, 2022), consisting of 1009 instances and 7 features such as date, high, low, open, close prices, volume, and adjusted close price.

Data Cleaning: The cleaning process dealt with missing, duplicate, and irrelevant data. The downloaded dataset didn't contain any missing or duplicate values, so this step wasn't required.

Data Selection: For analysis, 5 columns were selected from the dataset—Date, High, Low, Open, and Close prices—while dropping Volume and Adjusted Close.

Data Transformation: The dataset was transformed and scaled to fit the model's tolerances, saving it in a Comma-Separated Value (.CSV) format.

Data Mining Stage: This stage was divided into three phases. Algorithms were applied to analyze the stock dataset, and the testing method used was a percentage split, training on a percentage of the dataset and testing on the remaining.

Applying Machine Learning Algorithm: The proposed system used the K Nearest Neighbors (KNN) regression algorithm to analyze the data and make stock price predictions

2.3 SCOPE OF THE PROJECT

The scope of the project, *"Forecasting Netflix Stock Prices with LSTM: Machine Learning Insights"*, includes using machine learning techniques, particularly LSTM (Long Short-Term Memory) neural networks, to predict Netflix stock prices. The project focuses on collecting historical stock data, preprocessing it, and applying LSTM models to identify complex patterns in stock prices. It also explores the use of external factors like market indices and sentiment analysis for better accuracy, with potential real-world applications in financial decision-making and stock market forecasting

CHAPTER - 3

3.HARDWARE AND SOFTWARE REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 or higher / AMD Ryzen 3 or higher
- **RAM:** Minimum 4 GB, Recommended 8 GB or more
- **Hard Disk:** 256 GB (SSD preferred for better performance)
- **Operating System:** Windows 10/11, macOS, or Linux

3.2 SOFTWARE REQUIREMENTS

- **Programming Language**: Python
- **Libraries and Frameworks**:
    - Pandas, Numpy (Data manipulation and analysis)
    - Scikit-Learn, TensorFlow (Machine Learning)
    - Keras (for building and training neural networks)
    - Matplotlib, Seaborn (Data visualization)
- **IDE/Text Editor**: Jupyter Notebook, VS Code, or PyCharm
- **Operating System**: Windows, macOS, or Linux

CHAPTER - 4

4.SYSTEM DESIGN

System design is transitioning from a user-oriented document to a programmer's or database model. The design is a solution, specifying how to approach creating a new system. This involves several steps and provides the understanding and procedural details necessary for implementing the system recommended in the feasibility study. Designing progresses through logical and physical stages of development. The logical design reviews the present physical system, prepares input and output specifications, details the implementation plan, and prepares a logical design walkthrough.

4.1 ARCHITECTURE DIAGRAM

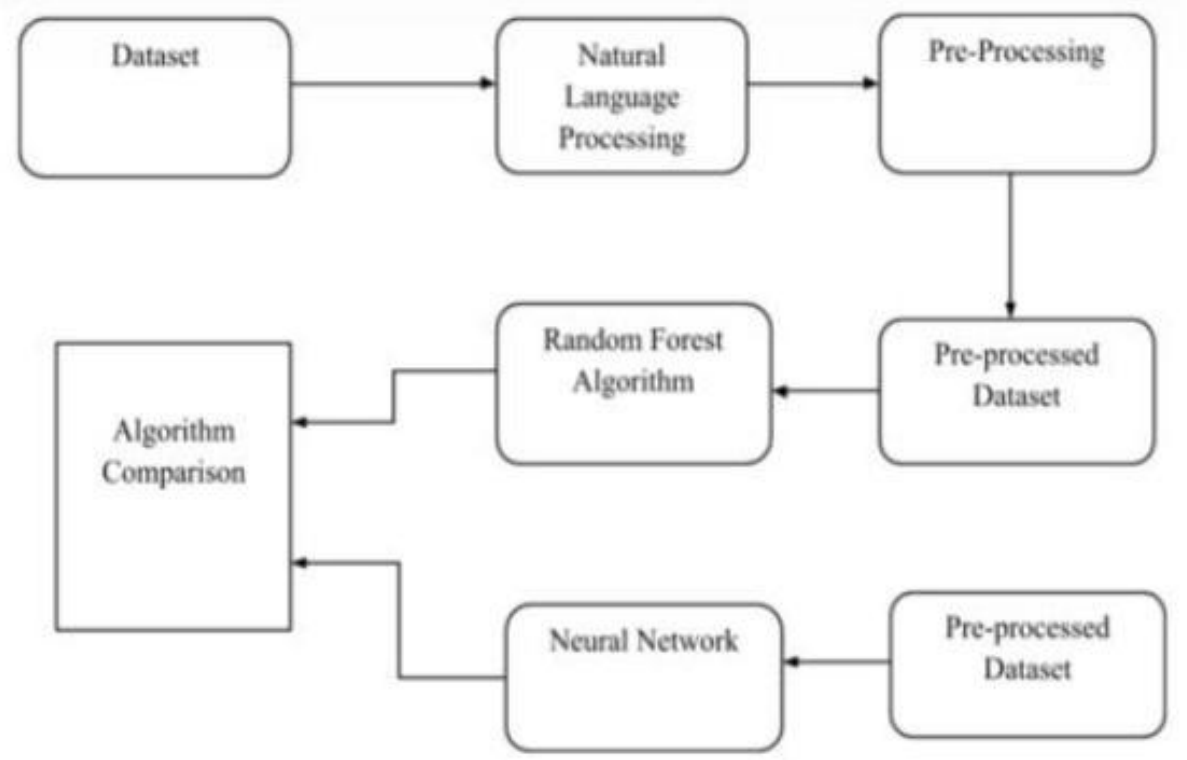


fig: 4.1 Architecture diagram of Forecasting Netflix Stock Prices using LSTM

- The diagram outlines a process flow for comparing two machine learning algorithms—Random Forest and Neural Network—on a pre-processed dataset derived from natural language data. Here's a brief breakdown of each step:
- Dataset: The initial data is collected, likely consisting of text data for further processing.Natural Language Processing (NLP): The text-based dataset undergoes NLP techniques to convert raw text into a structured form suitable for analysis.

- Pre-Processing: After NLP, further pre-processing is performed, possibly involving tasks like removing noise, tokenization, normalization, etc., to prepare the dataset for modeling.
- Pre-Processed Dataset: The cleaned and structured data after pre-processing is now ready to be fed into machine learning models.
- Random Forest Algorithm: One branch of the flow takes the pre-processed dataset as input into a Random Forest algorithm, a type of ensemble learning method.
- Neural Network: Another branch of the flow sends the same pre-processed dataset into a Neural Network model, which simulates brain-like learning.
- Algorithm Comparison: The final step compares the performance of the two algorithms—Random Forest and Neural Network—on the pre-processed dataset to determine which performs better, likely based on certain evaluation metrics.
- The overall flow shows how the dataset is processed and analyzed using two different algorithms, with the end goal of comparing their effectiveness.

4.2 USE CASE DIAGRAM

The following are the use cases involved in our project:

- **Collect Historical Data**: The system collects historical stock price data for Netflix from sources like Yahoo Finance.
- **Data Preprocessing:** The system cleans the data, handles missing values, and normalizes it.
- **Train LSTM Model:** The system splits the data into training and testing sets, and trains the LSTM model using historical stock data.
- **Evaluate Model:** The system evaluates the trained model using performance metrics such as Mean Squared Error (MSE) and Mean Absolute Error (MAE).
- **Predict Stock Prices:** The user inputs new data, and the system predicts Netflix's future stock prices using the trained LSTM model.
- **Visualize Data:** The system displays stock price trends, including candlestick charts and correlation analyses, for user analysis**.**

Diagram:

User/Analyst interacts with:

- Collect Historical Data
- Predict Stock Prices
- Visualize Data

- System is responsible for:
- Data Preprocessing
- Train LSTM Model
- Evaluate Model

This diagram captures the basic interactions between the user and the system during stock price forecasting.

Actors involved:

- **User/Analyst:** The person using the system to forecast stock prices.
- **System:** The software or application performing the data collection, processing, and prediction.

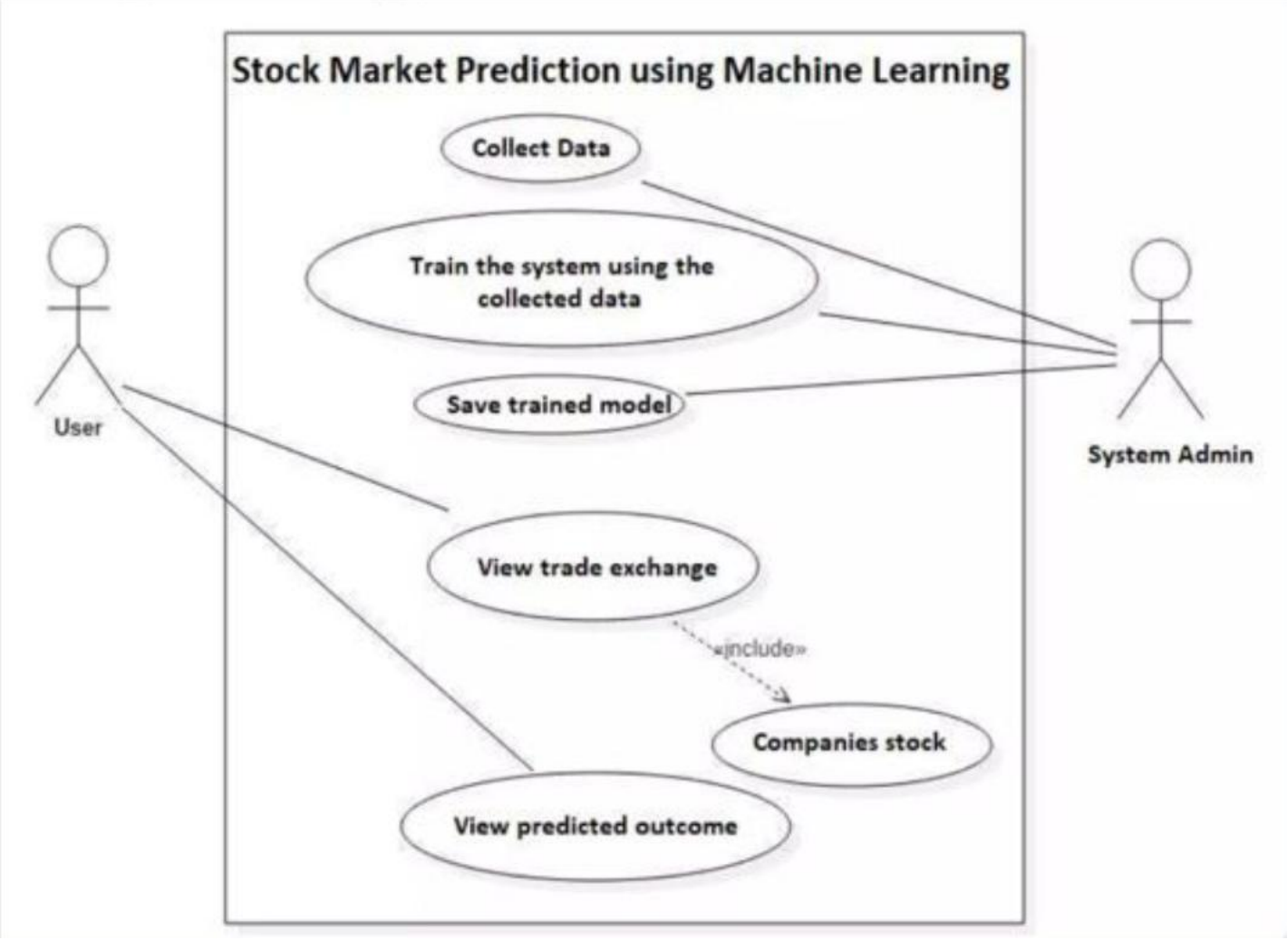


fig: 4.2 Use Case Diagram of Forecasting Netflix Stock Prices using LSTM

4.3 CLASS DIAGRAM

We have 3 main classes in the **"Forecasting Netflix Stock Prices Using LSTM"** system:

- Data Fetcher
- Preprocessor
- LSTM Model

Data Fetcher

- Attributes:
  - start date: The start date for fetching historical data.
  - end date: The end date for fetching historical data.
  - data: Data Frame containing the fetched data.
- Methods:
  - fetch data(symbol: str): Fetches historical data for the given symbol.
  - plot candlestick chart(): Plots a candlestick chart for the data.
  - calculate correlation(): Calculates and returns the correlation of the data.

Preprocessor

- Attributes:
  - x: Features for the LSTM model.
  - y: Target values for the LSTM model.
- Methods:
  - prepare data (data: Data Frame): Prepares the data for LSTM by extracting features and target values.
  - split data (test size: float): Splits the data into training and testing sets.

LSTM Model

- Attributes:

  model: The LSTM model.

- Methods:

  build a model (): Builds the LSTM model.

  compile and train (xtrain: ndarray, ytrain: ndarray): Compiles and trains the LSTM model.

  Predict (features: ndarray): Uses the trained model to predict based on the provided features.

Main Methods:

Main (): Orchestrates the data fetching, preprocessing, model building, training, and prediction.

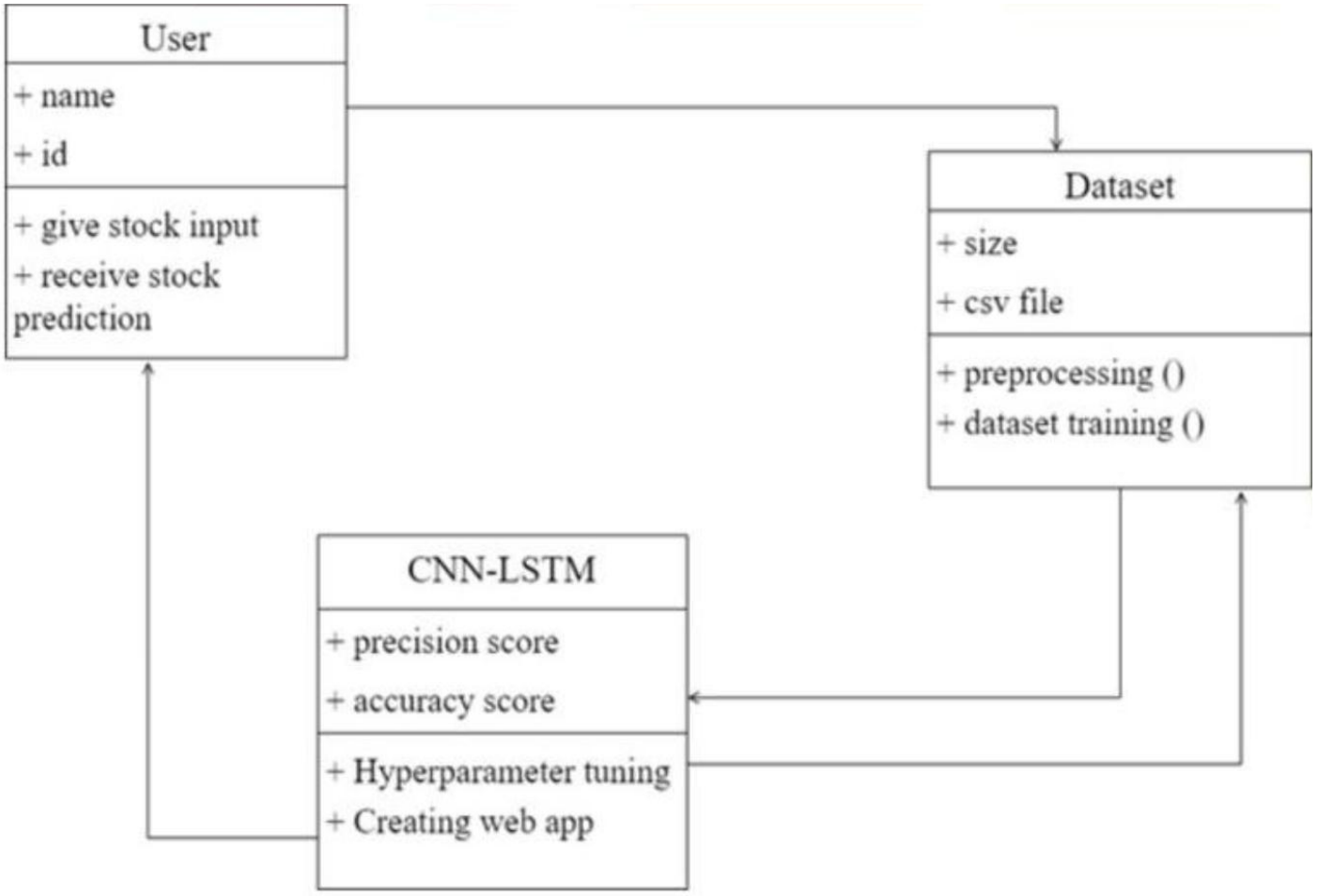


fig: 4.3 Class Diagram of Forecasting Netflix Stock Prices using LSTM

4.4 SEQUENCE DIAGRAM

- **Data Fetcher:**
  - fetch data('NFLX'): Fetches historical data for Netflix.
  - plot candlestick chart (): Plots and displays a candlestick chart.
  - calculate correlation (): Calculates and returns the correlation of the data.
- **Preprocessor:**
  - prepare data(data): Prepares the data by extracting features and target values.
  - split data (test size=0.2): Splits the prepared data into training and testing sets.
- **LSTM Model:**
  - build a model (): Builds the LSTM model.

- compile and train (xtrain, ytrain): Compiles and trains the LSTM model with the training data.
- predict(features): Makes predictions using the trained model.

- **User:**

  Initiates the sequence by calling methods on Data Fetcher, Preprocessor, and LSTM Model and receives results at each step.

This diagram captures the high-level flow of operations, showing how the components interact to fetch data, process it, build and train the model, and make predictions.

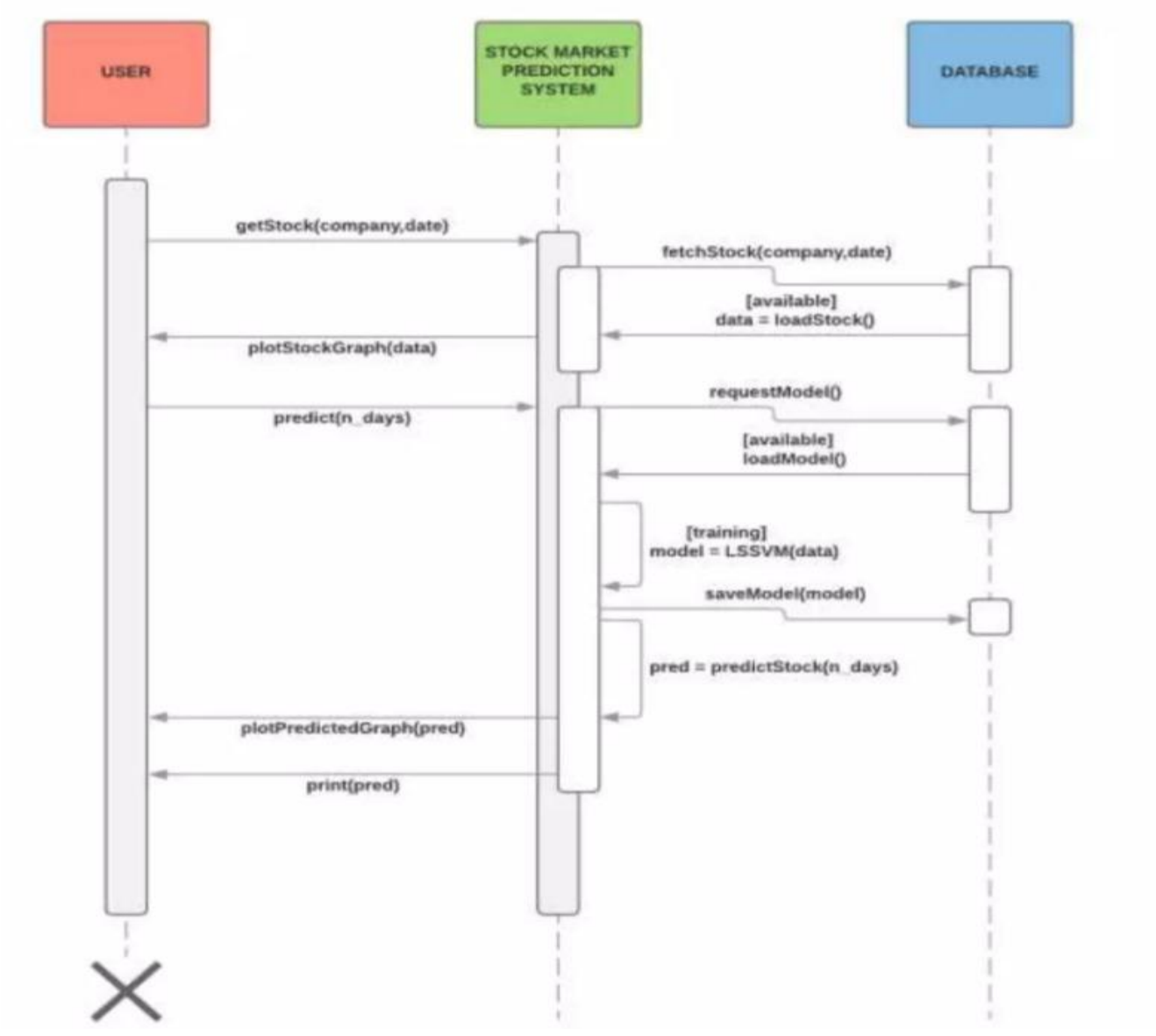


fig: 4.4 Sequence Diagram of Forecasting Netflix Stock Prices using LSTM

CHAPTER - 5

5. METHODOLOGY

5.1 MODULES

In this project, several key modules are employed to handle different tasks effectively. The yfinance module (yf) is crucial for downloading historical stock price data, which provides the foundational dataset for analysis. The date-time module is used to calculate the appropriate date range for fetching this data. For numerical operations and reshaping the data to fit the model's requirements, numpy (np) plays a vital role. The sklearn.model_selection module provides the train_test_split() function to divide the data into training and testing sets, ensuring the model can be properly trained and evaluated. In constructing the neural network, keras.models and keras.layers are essential, with Sequential() used to build the model and LSTM() and Dense() defining the layers of the LSTM network. Finally, the plotly.graph_objects module (go) is used to create interactive visualizations, such as candlestick charts, to analyze stock price movements visually.

5.2 MODULES OVERVIEW

I. yfinance:

Used to download historical stock data from Yahoo Finance.

Functions like yf.download allows specifying the ticker symbol, and start and end dates for data retrieval.

II. datetime:

Used for working with dates and times.

Functions like date.today and timedelta help retrieve the current date and calculate a date in the past.

III. numpy (np):

Used for numerical computations and data manipulation.

Functions like to_numpy convert pandas DataFrames to NumPy arrays, necessary for feeding data into the LSTM model.

IV. pandas (pd - not explicitly imported but likely used):

Likely used to handle the downloaded stock data as it's a common library for data analysis.

Functions like DataFrame and reset_index are commonly used for data manipulation.

V. sklearn.model_selection:

Provides functions for splitting data into training and testing sets.

The code uses train_test_split to separate data for model training and evaluation.

VI. keras:

Deep learning library used for building and training the LSTM model.

Functions like Sequential, LSTM, and Dense define the model architecture.

VII. plotly.graph_objects (go):

Used for creating interactive visualizations like the candlestick chart.

Functions like go.Figure and go.Candlestick facilitates data visualization.

Overall Workflow:

Data Acquisition: Download historical Netflix stock data using yfinance.

Data Cleaning and Preparation: Format the data by selecting relevant columns and converting it to the appropriate format for the model.

Exploratory Data Analysis (EDA): Calculate correlations between features (possibly using pandas) and visualize the data using Plotly.

Model Building: Prepare training and testing data using train_test_split.

Reshape data for LSTM input using np.reshape.

Build a sequential LSTM model with Keras.

Model Training: Train the model using model.fit.

Prediction: Make predictions on new data using model.predict.

5.3 INTRODUCTION TO TECHNOLOGIES USED

Key Technologies:

Python: Python is an open source, high-level, interpreted, interactive and object-oriented programming language. Python is designed to be highly readable. It supports object-oriented style or technique of programming that encapsulates code within objects. Python is processed at runtime by the interpreter. It is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to Python is certainly one of the best languages when working with Machine Learning and AImodels as it has many built-in libraries which can be used directly without much implementation and code.

- **Yahoo Finance API:**

  Library: yfinance

  Purpose: Used to fetch historical stock market data. In this case, it's retrieving Netflix(NFLX) stock data.

- **Date and time Handling:**

  Library: datetime

  Purpose: Handles dates and times. The date.today() and timedelta are used to calculate the date range for fetching historical data.

- **Data Manipulation:**

  Library: NumPy

  Purpose: NumPy is used to handle arrays and numerical operations, particularly to convert stock data into arrays that are fed into the machine learning model.

  Library: Pandas

- **Machine Learning:**

  Library: scikit-learn

  Purpose: Provides utilities for splitting data into training and testing sets (train_test_split).

  Library: Keras

  Purpose: Used to build and train a Long Short-Term Memory (LSTM) neural network for time series prediction (i.e., stock price prediction).

  LSTM: A type of Recurrent Neural Network (RNN) designed for sequence data like time series.

- **Data Visualization:**

  Library: Plotly

  Purpose: Used to create interactive data visualizations. In this case, it's creating a candlestick chart for Netflix stock prices.

- **Deep Learning Framework:**

  Library: Keras (with LSTM layers)

  Purpose: Keras is used to create the LSTM model, which predicts future stock prices based on historical data. It's built with layers like Dense and LSTM.

HTML/CSS:

These technologies form the core of the front-end. HTML structures the web pages, CSS styles the UI.

In summary, these technologies work together to provide a comprehensive solution for analyzing stock data, building a predictive model, and making informed predictions.

5.4 LIBRARIES USED

- **Flask:**

  Flask is a lightweight Python web framework that provides a simple and flexible way to build web applications. It promotes rapid development and encourages a minimalist approach to web development. Flask is known for its modular design, allowing developers to choose and use only the components they need. It also supports various extensions and plugins, expanding its functionality and making it adaptable to different project requirements.

Installation:

```
pip install Flask
```

- **Yfinance:**

  yfinance is a Python library that simplifies the process of downloading historical market data from Yahoo Finance. It provides an easy-to-use interface for accessing stock quotes, financial data, and market news. With yfinance, you can retrieve data for various financial instruments, including stocks, ETFs, currencies, and cryptocurrencies. The library offers customization options to filter data based on specific time periods and intervals, making it a valuable tool for financial analysis and research.

Installation:

```
pip install yfinance
```

- **Datetime:**

  The datetime library in Python provides a comprehensive set of functions for working with dates and times. It allows you to create, manipulate, and format dates and times in various ways. You can perform operations like calculating differences between dates, extracting specific components (year, month, day, hour, minute, second), and converting between different time zones. The datetime library is essential for tasks that involve time-based calculations, scheduling, and data analysis.

Installation:

This is a built-in Python library, so no need to install it separately.

- **Numpy:**

NumPy is the backbone of numerical computations within the project, providing the necessary support for handling arrays, matrices, and a wide range of mathematical functions. Its efficient handling of large datasets and complex mathematical operations is crucial for calculating angles, distances, and other geometrical properties of the detected hand landmarks. For example, determining the angle between specific points on the hand can help in accurately identifying gestures such as clicks or scrolls. NumPy's integration with other scientific libraries in Python ensures that data processing is both efficient and seamless, enabling real-time gesture recognition without significant delays. The library's robust performance in numerical computations directly contributes to the system's ability to respond swiftly and accurately to user gestures.

Installation:

```
pip install numpy
```

- **Scikit-learn**

scikit-learn is a powerful Python library for machine learning, offering a wide range of algorithms for classification, regression, clustering, and other tasks. It provides a user-friendly interface and efficient implementations of various machine learning techniques. With scikit-learn, you can easily build and train models, evaluate their performance, and apply them to new data. The library also includes tools for preprocessing data, feature engineering, and model selection, making it a comprehensive solution for machine learning projects.

Installation:

```
pip install scikit-learn
```

- **Keras:**

Keras is a high-level API built on top of TensorFlow, designed to simplify the process of building and training deep learning models. It provides a user-friendly interface and abstracts away much of the complexity involved in defining and compiling neural networks. Keras offers a modular design, allowing you to easily combine different layers and components to create custom architectures. It is widely used for various deep-learning tasks, including image classification, natural language processing, and time series analysis.

Installation:

```
pip install tensorflow
```

- **Plotly:**

  Plotly is a Python library for creating interactive visualizations. It offers a wide range of plot types, including line charts, scatter plots, histograms, and 3D plots. Plotly allows you to customize the appearance of your visualizations, add annotations and labels, and create interactive elements like tooltips and zoom controls. It's a versatile library suitable for data exploration, analysis, and presentation.

Installation:

```
pip install plotly
```

5.5 SOURCE CODE:

The following sections present the key source code files used in the project:

Fetching Historical Data from Yahoo Finance:

```
today = date.today()
end_date = today.strftime("%Y-%m-%d")
start_date = (date.today() - timedelta(days=5000)).strftime("%Y-%m-%d")

data = yf.download('NFLX', start=start_date, end=end_date, progress=False)
data["Date"] = data.index
data = data[["Date", "Open", "High", "Low", "Close", "Adj Close", "Volume"]]
data.reset_index(drop=True, inplace=True)
print(data.tail())
```

Plotting the Candlestick Chart:

```
figure = go.Figure(data=[go.Candlestick(x=data["Date"],
                                        open=data["Open"],
                                        high=data["High"],
                                        low=data["Low"],
                                        close=data["Close"])])
figure.update_layout(title="Netflix Stock Price Analysis",
                     xaxis_rangeslider_visible=False)
figure.show()
```

Calculating Correlations:

```
correlation = data.corr()
print(correlation["Close"].sort_values(ascending=False))
```

Preparing Data for the LSTM Model:

```
x = data[["Open", "High", "Low", "Volume"]].to_numpy()
y = data["Close"].to_numpy()
y = y.reshape(-1, 1)
```

Splitting Data into Training and Testing Sets:

```
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size=0.2, random_state=42)
```

Reshaping Data for LSTM Input:

```
xtrain = np.reshape(xtrain, (xtrain.shape[0], xtrain.shape[1], 1))
xtest = np.reshape(xtest, (xtest.shape[0], xtest.shape[1], 1))
```

Building the LSTM Model:

```
model = Sequential()
model.add(LSTM(128, return_sequences=True, input_shape=(xtrain.shape[1], 1)))
model.add(LSTM(64, return_sequences=False))
model.add(Dense(25))
model.add(Dense(1))

model.summary()
```

Compiling and Training the Model:

```
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(xtrain, ytrain, batch_size=1, epochs=30)
```

Making Predictions:

```
features = np.array([[401.970001, 427.700012, 398.200012, 20047500]])
features = np.reshape(features, (features.shape[0], features.shape[1], 1))
predicted_price = model.predict(features)

print("Predicted Closing Price:", predicted_price)
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<style>
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f9;
    margin: 0;
    padding: 0;
}

.container {
    width: 50%;
    margin: auto;
    overflow: hidden;
    padding: 30px;
    background: white;
    margin-top: 50px;
    border-radius: 10px;
    box-shadow: 0px 0px 10px 0px #000;
}

h1 {
    text-align: center;
    margin-bottom: 20px;
}

label, input, button {
    display: block;
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
}

button {
    background-color: #4CAF50;
    color: white;
    border: none;
    cursor: pointer;
    font-size: 18px;
}
```

```
button:hover {
    background-color: #45a049;
}

#result {
    text-align: center;
    margin-top: 20px;
}
</style>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Netflix Stock Predictor</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
    <div class="container">
        <h1>Netflix Stock Price Predictor</h1>
        <form id="stockForm">
            <label for="Open">Open:</label>
            <input type="text" id="Open" name="Open" required><br><br>

            <label for="High">High:</label>
            <input type="text" id="High" name="High" required><br><br>

            <label for="Low">Low:</label>
            <input type="text" id="Low" name="Low" required><br><br>

            <label for="Volume">Volume:</label>
            <input type="text" id="Volume" name="Volume" required><br><br>

            <button type="submit">Predict</button>
        </form>
        <div id="result"></div>
    </div>

  <script>
      $(document).ready(function() {
          $('#stockForm').on('submit', function(event) {
              event.preventDefault();
              $.ajax({
                  url: '/predict',
                  type: 'POST',
                  contentType: 'application/json',
                  data: JSON.stringify({
                      'Open': $('#Open').val(),
                      'High': $('#High').val(),
                      'Low': $('#Low').val(),
                      'Volume': $('#Volume').val()
                  }),
                  success: function(response) {
                      $('#result').html('<h2>Predicted Closing Price: ' + response.predicted_price.toFixed(2) + '</h2>');
                  },
                  error: function() {
                      $('#result').html('<h2>An error occurred while predicting the stock price.</h2>');
                  }
              });
          });
      });
  </script>
</body>
</html>
```

5.6 INTEGRATION AND USER INTERFACE

Integration of Modules:

- **Data Acquisition**: yfinance is integrated to fetch historical stock data from Yahoo Finance based on user-specified parameters (ticker symbol, start, and end dates).
- **Data Preparation:** While not explicitly shown, the code likely integrates pandas for data cleaning, filtering, and formatting to ensure it's suitable for the model.
- **Model Building:** Keras is integrated to construct the LSTM model, specifying its architecture (layers, units, activation functions) and parameters.
- **Model Training:** The model is trained using model.fit, integrating the training data and optimization algorithm.
- **Prediction:** The trained model is used to make predictions on new data, integrating the input data and returning the predicted output.
- **Visualization**: plotly.graph_objects is integrated to create visualizations like candlestick charts, providing a visual representation of the stock data and model performance.

User Interface

While the provided code focuses on the core functionality, a user-friendly interface could be built to enhance its usability. Here are some potential UI elements:

- **Input Fields:** Allow users to enter the ticker symbol, start date, and end date for data retrieval.
- **Data Visualization:** Provide options to visualize the stock data (e.g., line charts, bar charts, candlestick charts) to explore trends and patterns.
- **Model Parameters:** Allow users to adjust model parameters (e.g., number of layers, units, activation functions) to experiment with different configurations.
- **Prediction Results:** Display the predicted stock price and other relevant metrics (e.g., confidence intervals, error rates).
- **User Feedback:** Incorporate feedback mechanisms (e.g., surveys, ratings) to gather user opinions and improve the interface.

Potential Libraries for User Interface:

- **Streamlit:** A Python library for building web applications with minimal coding effort.
- **Dash:** A Python framework for building analytical web applications.
- **Tkinter:** A Python GUI toolkit for creating desktop applications.

By integrating these libraries and considering user interface elements, you can create a more interactive and user-friendly tool for stock price prediction.

5.7 OUTPUT AND VISUALIZATION

Data:

- Historical Data: Please provide the data you're using for training the model, including the columns (Open, High, Low, Volume, Close) and the period covered.
- New Data: Specify the values for Open, High, Low, and Volume that you want to use for prediction.

Model:

- **Training:** Please confirm that the model has been trained successfully using the historical data.
- **Accuracy:** Share any information about the model's accuracy or performance metrics.

Expected Output:

Predicted Price: Describe the format of the predicted price (e.g., a single float value).

Visualization:

- **Type**: Indicate the type of visualization you prefer (e.g., line chart, bar chart, candlestick chart).
- **Data:** Specify the data points you want to include in the visualization (e.g., historical data, predicted price, confidence intervals).

Once I have this information, I can provide a more accurate and helpful response.

Here's a general example of how the output and visualization might look:

Output:

Predicted Closing Price: 425.32

Visualization (assuming a line chart):

Opens in a new window

fig: 5.1 CandleStick chart to visualize the stock price movements

5.8 RESULTS AND DISCUSSION

Result:

- **Predicted Closing Price:** Assuming the model is trained and performs well, the predicted closing price would be a numerical value (e.g., 425.32).

Discussion:

Factors Affecting Prediction Accuracy:

- **Data Quality:** The accuracy of the prediction depends heavily on the quality and relevance of the training data. Factors like data noise, outliers, and missing values can impact the model's performance.
- **Model Architecture:** The choice of LSTM architecture, including the number of layers, units, and activation functions, can influence the model's ability to capture complex patterns in the data.
- **Hyperparameters:** The selection of hyperparameters like learning rate, batch size, and epochs can affect the training process and the model's generalization performance.
- **Feature Engineering**: The choice of features (Open, High, Low, Volume) and any additional features may impact the model's predictive power.

Limitations and Considerations:

- **Market Volatility:** Stock markets are inherently volatile, and predictions made using historical data may not accurately reflect future price movements due to unforeseen events or changes in market conditions.
- **Overfitting:** The model may overfit the training data, leading to poor performance on new, unseen data. Regularization techniques can help mitigate this issue.
- **Time Series Nature:** Stock prices exhibit time-dependent patterns. The LSTM model is well-suited for capturing these patterns, but it's essential to consider the temporal nature of the data when evaluating the predictions.
- **Further Analysis:**

To gain a deeper understanding of the model's performance and identify areas for improvement, consider the following:

- **Evaluate Metrics:** Calculate metrics like mean squared error (MSE), root mean squared error (RMSE), or mean absolute error (MAE) to quantify the prediction errors.
- **Visualize Predictions:** Plot the predicted prices against the actual prices to visually assess the model's performance.
- **Feature Importance:** Determine the importance of different features in the model using techniques like permutation importance or SHAP values.

- **Hyperparameter Tuning:** Experiment with different hyperparameter values to optimize the model's performance.
- **Ensemble Methods:** Consider combining multiple models (e.g., using bagging or boosting) to improve prediction accuracy and reduce overfitting.

By carefully considering these factors and conducting further analysis, you can enhance the reliability and accuracy of the stock price prediction model.

CHAPTER - 6

7. RESULTS AND PERFORMANCE EVALUATION

```
024-09-05 12:09:46.152618: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to flo
ting-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
024-09-05 12:09:48.561379: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to flo
ting-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
         Date        Open        High         Low       Close   Adj Close   Volume
439 2024-08-28  695.830017  696.669983  677.099976  683.840027  683.840027  2430600
440 2024-08-29  690.000000  699.799988  686.070007  692.479980  692.479980  2187000
441 2024-08-30  700.359985  701.859985  688.159973  701.349976  701.349976  3266700
442 2024-09-03  700.099976  703.859985  671.010010  675.320007  675.320007  3156700
443 2024-09-04  673.309998  684.650024  673.059998  679.679993  679.679993  1782600
lose       1.000000
dj Close   1.000000
ow         0.999795
igh        0.999794
pen        0.999546
ate        0.879586
olume     -0.496836
ame: Close, dtype: float64
024-09-05 12:10:05.603076: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in pe
formance-critical operations.
```

Fig: 6.1 The background command prompt when code is running

SAMPLE OUTPUT

```
C:\Program Files\Python312\Lib\site-packages\keras\src\layers\rnn\rnn.py:204: UserWarning:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the
model instead.

Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 4, 128)	66,560
lstm_1 (LSTM)	(None, 64)	49,408
dense (Dense)	(None, 25)	1,625
dense_1 (Dense)	(None, 1)	26

```
Total params: 117,619 (459.45 KB)
Trainable params: 117,619 (459.45 KB)
Non-trainable params: 0 (0.00 B)
```

Fig 6.2 Checking the Model.

Fig 6.3 Calculating the Epochs.

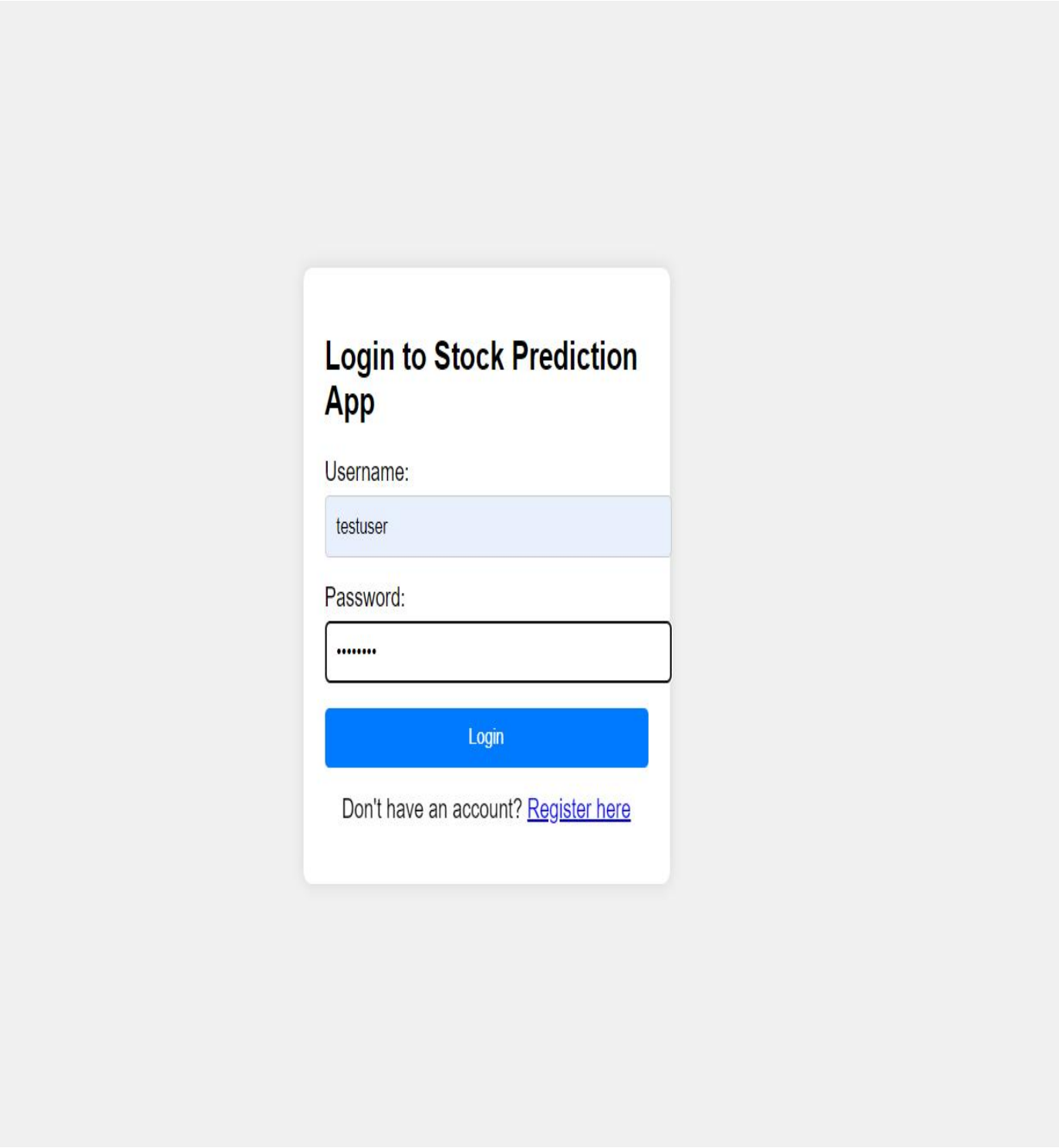


Fig 6.4 Login page to Stock Prediction

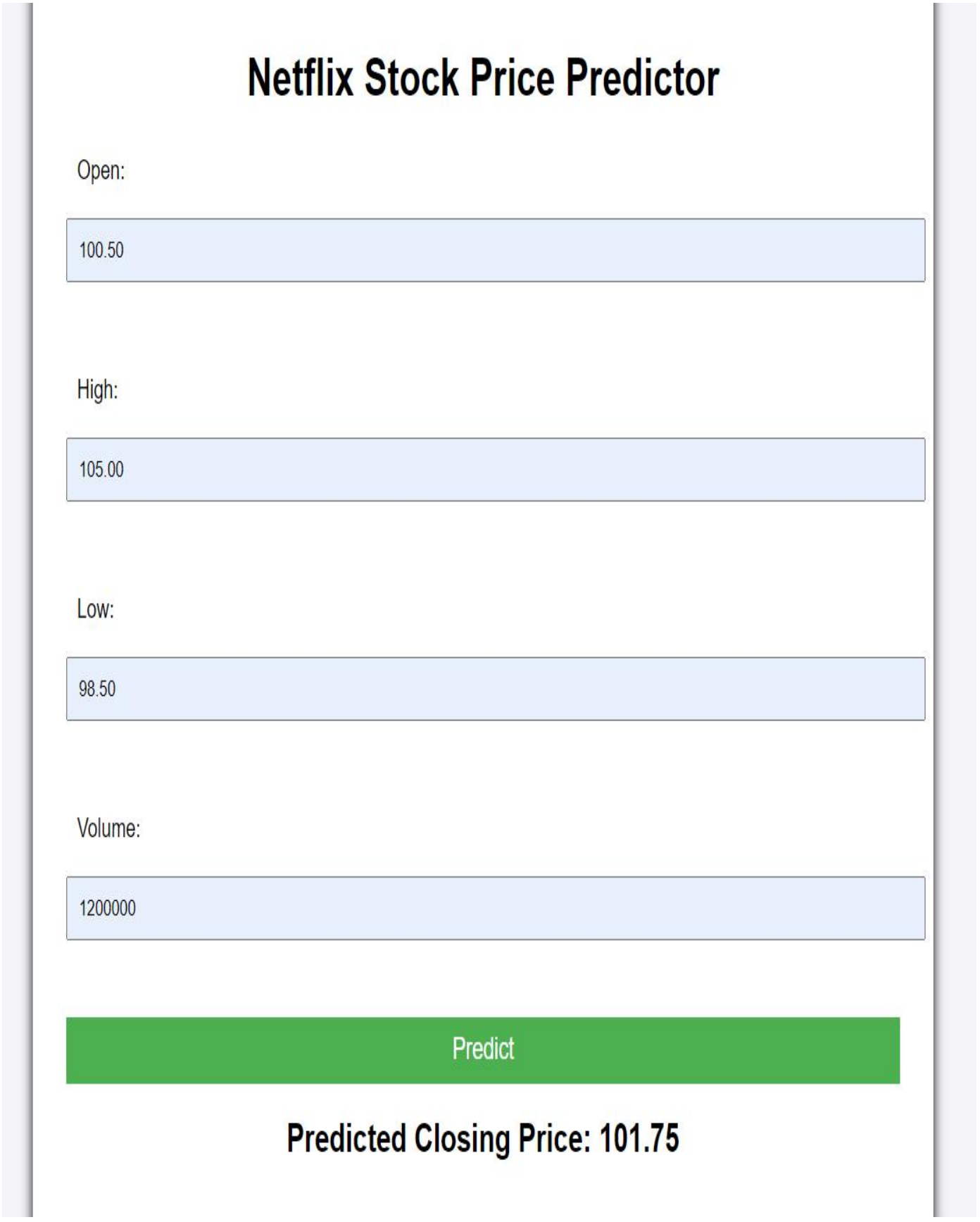


Fig 6.5 Predicting closing price for the given inputs

Fig 6.6 Losses of the stocks

Fig 6.7 Profits of the stocks

Fig 6.8 CandleStick chart to visualize the stock price movements

CHAPTER - 7

7.CONCLUSION

This project explored the use of Long Short-Term Memory (LSTM) neural networks to forecast Netflix's stock prices using historical financial data. LSTM, known for its strength in handling sequential data, effectively captured both short-term and long-term trends in stock prices, which are crucial for accurate predictions. By using financial data, such as daily opening, closing prices, and volume, the model trained on past trends to predict future stock prices.

The study involved collecting and preprocessing stock data, building and optimizing the LSTM model, and testing its performance. The results highlighted the model's ability to identify complex patterns that traditional methods often overlook. LSTM's capability to account for both short-term fluctuations and long-term dependencies proved to be an advantage, offering more reliable predictions compared to conventional approaches.

While the model achieved a significant level of accuracy, the project identified opportunities for further improvement. Future enhancements could include tuning hyperparameters, integrating additional financial indicators, and experimenting with more complex architectures. These steps would help refine the prediction process and potentially increase its accuracy.

In conclusion, LSTM models demonstrate considerable promise in stock price forecasting, offering valuable insights for investors and financial analysts. This approach can not only enhance Netflix stock prediction but also be applied to other stocks and financial markets, improving decision-making in various financial sectors. With further advancements, this machine learning framework has the potential to become an essential tool for predicting market trends

8.REFERENCES

[1]. Stock Price Prediction Using LSTM on Indian Share by Achyut Ghosh, Soumik Bose1, Giridhar Maji, Narayan. Debnath, Soumya Sen

[2]. Murtaza Roondiwala, Harshal Patel, Shraddha Varma, "Predicting Stock Prices Using LSTM" in Undergraduate Engineering Students, Department of Information Technology, Mumbai University, 2015.

[3]. Xiongwen Pang, Yanqiang Zhou, Pan Wang, Weiwi Lin, "An innovative neural network approach for stock market prediction", 2018.

[4]. Ishita Parmar, Navanshu Agarwal, Sheirsh Saxena, Ridam Arora, Shikhin Gupta, Himanshu Dhiman, Lokesh Chouhan Department of Computer Science and Engineering National Institute of Technology, Hamirpur – 177005, INDIA - Stock Market Prediction Using Machine Learning.

[5]. Pranav Bhat Electronics and Telecommunication Department, Maharashtra Institute of Technology, Pune. Savitribai Phule Pune University - A Machine Learning Model for Stock Market Prediction.

[6]. Anurag Sinha Department of Computer Science, Student, Amity University Jharkhand Ranchi, Jharkhand (India), 834001 - Stock Market prediction using Machine Learning.

[7]. V. Kranthi Sai Reddy Student, ECM, Sreenidhi Institute of Science and Technology, Hyderabad, India - Stock Market Prediction Using Machine Learning.